DESIGN & ANALYSIS OF ALGORITHMS

ORIGINAL QUESTION BANK — MODULE-WISE

All questions are retained in their original wording.

Only the arrangement into five modules and the section headings have been changed. No question has been deleted, rewritten, shortened, corrected, or merged.

SHORT ANSWER QUESTIONS • ESSAY QUESTIONS

Source: uploaded dingudinga.txt

MODULE 1 — Algorithm Analysis, Recurrences & Divide and Conquer

SHORT ANSWER — 15 QUESTIONS

1. What is algorithm analysis? Define time complexity and space complexity.
2. Define Best Case, Average Case, and Worst Case analysis of an algorithm.
3. List and define the following common complexity classes: O(1), O(log n), O(n), O(n log n), and O(n^2).
4. What is asymptotic analysis? Explain the need for using asymptotic notations in algorithm analysis.
5. Define Big-O, Big-Ω (Omega), and Big-Θ (Theta) notations. State what type of bound each notation represents.
6. State any four important properties of asymptotic notations.
7. What is a recurrence equation? Explain its role in analyzing recursive algorithms with a suitable example.
8. Explain how a recurrence equation is formed for a recursive algorithm. Illustrate with an example.
9. State the Substitution Method for solving recurrence equations and list the basic steps involved
10. What is the Iteration Method for solving recurrence equations? Explain the basic steps involved
11. Define Divide and Conquer. Explain its three basic steps with a suitable example.
12. What is Control Abstraction in Divide and Conquer? Write the general structure of a Divide and Conquer algorithm.
13. What is Merge Sort? Explain its basic working and state its time complexity.
14. Explain the partitioning process in Quick Sort and state its best, average, and worst-case time complexities.
15. Differentiate between conventional matrix multiplication and the Divide and Conquer approach to matrix multiplication

ESSAY — 10 QUESTIONS

1. Explain the different types of complexity analysis: time complexity, space complexity, best case, average case, and worst case, with suitable examples.
2. Explain the common complexity classes O(1), O(log n), O(n), O(n log n), and O(n^2). Compare their growth rates and give one suitable algorithm example for each.
3. Explain Big-O, Big-Ω, and Big-Θ notations with their formal definitions, significance, and suitable examples
4. Determine the asymptotic complexity of the following functions and compare their growth rates: f(n) = $3n^2 + 5n + 7$, g(n) = n log n, and h(n) = 2■. T(n) = 2T(n/2) + n, with T(1) = 1, using the Substitution Method
5. Solve the recurrence T(n) = 2T(n/2) + n, with T(1) = 1, using the Substitution Method
6. Formulate and solve the recurrence equation for an algorithm that divides an input of size n into two equal parts, recursively processes both parts, and spends n units of time to combine the results. Use the Substitution Method
7. Solve the recurrence T(n) = T(n/2) + n, with T(1) = 1, using the Iteration Method.
8. Explain the Divide and Conquer strategy, Control Abstraction, general structure, advantages, and applications with a suitable example.
9. Explain the Merge Sort algorithm with its working and analyze its time and space complexity
10. Explain Quick Sort with its algorithm, partitioning process, and best, average, and worst-case complexity

MODULE 2 — Greedy Strategy & Dynamic Programming

SHORT ANSWER — 15 QUESTIONS

1. Define the Greedy Strategy. Explain any three characteristics of the Greedy approach.
2. Define the Greedy Choice Property? Explain its significance in designing a greedy algorithm.
3. List and briefly explain the main components of the Control Abstraction for a Greedy Algorithm.
4. Explain the Fractional Knapsack Problem. How does the greedy approach select items to maximize the total profit
5. State the steps of the greedy algorithm for the Fractional Knapsack Problem and explain its time complexity.
6. Define the Job Scheduling with Deadlines problem. Explain the greedy criterion used for selecting and scheduling jobs.
7. Define a Minimum Spanning Tree (MST). Explain any three important properties of an MST.
8. Explain the working principle of Prim's Algorithm and state its time complexity
9. Explain the working principle of Kruskal's Algorithm. How does it avoid creating cycles
10. Define Dynamic Programming. Explain the two main characteristics of problems suitable for Dynamic Programming.
11. Differentiate between Greedy Strategy and Dynamic Programming with respect to decision making, subproblems, optimality, and reuse of results.
12. Explain Memoization and Tabulation as approaches to Dynamic Programming.
13. Explain the Floyd–Warshall approach for solving the All-Pairs Shortest Path problem.
14. Define the Travelling Salesman Problem (TSP). Explain how Dynamic Programming can be used to solve TSP
15. Define the Bellman–Ford Algorithm? Explain how it handles negative edge weights and detects negative-weight cycles

ESSAY — 10 QUESTIONS

1. Explain the procedure for proving the correctness of a Greedy Algorithm. Illustrate the role of the Greedy Choice Property and Optimal Substructure.
2. Explain the Greedy Strategy in detail. Discuss its characteristics, Greedy Choice Property, Control Abstraction, correctness, and major applications.
3. Solve the following Fractional Knapsack problem using the greedy approach. There are four items with weights (10, 20, 30, 40) and profits (60, 100, 120, 160), respectively. The capacity of the knapsack is 50. Show the selection process and calculate the maximum profit.
4. Consider the following jobs: J1(deadline = 2, profit = 100), J2(deadline = 1, profit = 19), J3(deadline = 2, profit = 27), J4(deadline = 1, profit = 25), and J5(deadline = 3, profit = 15). Using the greedy Job Scheduling with Deadlines algorithm, determine the optimal job sequence and maximum profit.
5. For the weighted graph with vertices {A, B, C, D, E} and edges AB=2, AC=3, BC=1, BD=4, CD=5, CE=6, DE=2, use Prim's Algorithm starting from vertex A to construct the Minimum Spanning Tree. Show the edges selected at each step and calculate the total MST weight.
6. Apply Kruskal's Algorithm to the graph with vertices {A, B, C, D, E} and edges AB=2, AC=3, BC=1, BD=4, CD=5, CE=6, DE=2. Show the sorted edges, identify the edges accepted/rejected, construct the MST, and compare the result with Prim's Algorithm.
7. Explain the Control Abstraction for Dynamic Programming. Discuss the Principle of Optimal Substructure and Overlapping Subproblems with suitable examples.
8. Consider the Fibonacci sequence defined by F(0)=0, F(1)=1, and F(n)=F(n−1)+F(n−2). Explain how Dynamic Programming can be used to calculate F(6) using tabulation. Show the table and compare its efficiency with the naïve recursive approach.
9. Explain the Travelling Salesman Problem using Dynamic Programming. Describe the problem formulation
10. Explain the Bellman–Ford Algorithm for the Single-Source Shortest Path problem. Discuss its algorithm, relaxation process, handling of negative edge weights, negative-cycle detection, and complexity with a suitable example.

MODULE 3 — Backtracking, Branch and Bound & Lower Bounds

SHORT ANSWER — 15 QUESTIONS

1. Apply the backtracking technique to explain how a solution is constructed and abandoned when a partial solution cannot lead to a valid complete solution. Illustrate with a suitable example.
2. Given a state space tree for a backtracking problem, apply the concept of pruning to identify which branches should not be explored. Explain why these branches can be discarded.
3. Apply the general backtracking algorithm to describe how a solution is searched using a state space tree. Explain the roles of promising and non-promising nodes.
4. Apply the backtracking approach to formulate the N-Queens problem. State the constraints that must be satisfied when placing queens on an N×N chessboard.
5. Apply backtracking to represent the N-Queens problem as a state space tree. Explain what each level, node, and branch represents.
6. Apply the N-Queens backtracking algorithm to determine whether the partial placement [1, 3] is promising for a 4-Queens problem. Explain your answer.
7. Apply the backtracking technique to formulate the Sum of Subsets problem. State the conditions that must be satisfied by a valid subset.
8. Given the set S={5,10,12,13,15,18} and target sum M=30, apply backtracking to identify one valid subset whose sum is 30. Show the decisions made during the search.
9. Apply the Sum of Subsets backtracking method to explain how a state space tree is constructed. What do the left and right branches represent.
10. Apply the Branch and Bound technique to explain how a state space tree is constructed for a problem. Explain the roles of branching and bounding functions.
11. For an 8-puzzle state, apply a bounding function based on the number of misplaced tiles. Explain how the bound is used to select the next node for expansion.
12. Apply Branch and Bound to the 8-puzzle. Given the current state [1234567_8] and goal state [12345678_], determine the next move and explain the corresponding bound.
13. Apply the decision tree method to explain how a lower bound can be established for a comparison-based algorithm.
14. Apply the decision tree method to comparison-based sorting. Why is the lower bound for comparison-based sorting Ω(nlog■n)
15. Apply the decision tree method to comparison-based searching. Explain the lower bound for searching for an element in an unsorted collection of n elements.

ESSAY — 10 QUESTIONS

1. Apply the backtracking method to the 4-Queens problem. Explain how the state space tree is explored and how invalid placements are pruned.
2. Apply the general backtracking algorithm to explain the construction of a state space tree. Discuss the control abstraction, promising function, pruning, and major applications of backtracking.
3. Apply the backtracking algorithm to solve the 4-Queens problem. Show the sequence of queen placements and explain how invalid branches are pruned to obtain a valid solution.
4. Apply the general backtracking technique to develop the algorithm/pseudocode for the N-Queens problem. Explain the state space representation, promising condition, recursive process, and complexity
5. Using backtracking, solve the Sum of Subsets problem for S={3,5,6,7} with target sum M=12. Construct the search process, identify the valid subsets, and explain where pruning occurs.
6. Apply the backtracking technique to develop the algorithm/pseudocode for the Sum of Subsets problem. Explain the state space tree, promising condition, recursive process, and complexity with a suitable example.
7. Apply the Branch and Bound method to solve the following 8-puzzle problem. Show the state-space expansion and explain how the bounding function helps in selecting promising states. Initial state: [1235_6478], Goal state: [12345678_]. Use the number of misplaced tiles as the bounding function.

8. Apply the Branch and Bound technique to develop a solution strategy for the 8-Puzzle problem. Explain the control abstraction, state space tree, bounding function, node selection, pruning, and complexity considerations.
9. Apply the decision tree method to derive the lower bound for comparison-based sorting. Consider n distinct elements and explain why any comparison-based sorting algorithm requires at least log■2(n!) comparisons in the worst case
10. Apply the decision tree method to compare the lower bounds of comparison-based sorting and searching. Explain how the number of possible outcomes determines the required number of comparisons.

MODULE 4 — Complexity Theory & Network Flow

SHORT ANSWER — 15 QUESTIONS

1. Explain the concept of polynomial time in complexity theory. What is meant by a polynomial-time algorithm? Give one suitable example.
2. Differentiate between Class P and Class NP. State the defining property of each class and explain their relationship.
3. Consider a problem for which a proposed solution can be verified in polynomial time, but no polynomial-time algorithm for finding the solution is known.
4. Explain which complexity class the problem belongs to and why Define polynomial-time reduction. Explain the notation $A \leq_p B$ and state what it means for problem A to be polynomial-time reducible to problem B.
5. Explain the concept of reduction in computational complexity. Why are reductions useful when comparing the difficulty of two computational problems
6. Suppose problem A can be polynomial-time reduced to problem B. If B has a polynomial-time algorithm, explain what can be concluded about A. Justify your answer.
7. Define NP-Hard and NP-Complete problems. State one key difference between the two classes.
8. Explain the relationship among the classes P, NP, NP-Hard, and NP-Complete. Use a suitable representation or description to show their relationship.
9. A decision problem is known to be in NP, and a known NP-Complete problem can be polynomial-time reduced to it. What can you conclude about the problem? Explain.
10. Define the Vertex Cover problem. Given a graph $G=(V,E)$, explain what it means for a subset $C \subseteq V$ to be a vertex cover.
11. Define the Clique problem. What condition must a subset of vertices satisfy to form a clique? Illustrate with a suitable example.
12. Explain the relationship between Vertex Cover and Clique. For a graph $G=(V,E)$, state how a clique in G is related to a vertex cover in the complement graph G'.
13. Define a flow network and a network flow. State the main conditions that a valid flow must satisfy.
14. State the Max-Flow Min-Cut Theorem. Explain what is meant by a maximum flow and a minimum cut in a flow network
15. Explain the role of an augmenting path and a residual network in the Ford-Fulkerson method.

ESSAY — 10 QUESTIONS

1. Explain Class P and Class NP in detail. Discuss polynomial-time algorithms, polynomial-time verification, the relationship $P \subseteq NP$, and the significance of the P versus NP question. Give suitable examples of problems belonging to P and NP.
2. Compare problems in P and NP using suitable examples. Explain how polynomial-time solving differs from polynomial-time verification, and analyze why the distinction between P and NP is important in computer science.
3. Explain polynomial-time reductions in detail. Discuss the concept of reduction, polynomial-time reducibility, the notation $A \leq_p B$, and how reductions can be used to compare the computational difficulty of problems.
4. Explain the relationship between polynomial-time reductions and NP-Completeness. How are polynomial-time reductions used to prove that a problem is NP-Complete? Illustrate the procedure with a suitable example
5. Explain NP-Hard and NP-Complete problems in detail. Differentiate between them and discuss their relationship with P and NP. Give suitable examples of NP-Complete problems.
6. A new decision problem X is proposed. Describe the steps required to prove that X is NP-Complete using polynomial-time reductions. Explain how the result relates to the classes P, NP, and NP-Hard.
7. Explain the Vertex Cover and Clique problems in detail. Give their formulations, illustrate each with a suitable graph example, and explain the relationship between the two problems using the complement of a graph.
8. Explain why Vertex Cover and Clique are NP-Complete problems. Describe the role of polynomial-time verification and polynomial-time reductions in establishing their NP-Completeness
9. Explain the Ford-Fulkerson method for solving the Maximum Flow Problem. Discuss residual networks and augmenting paths, and describe the algorithm step-by-step with a suitable example.

10. State and explain the Max-Flow Min-Cut Theorem. Using the theorem, explain how a minimum cut can be used to establish that a flow is maximum. Illustrate your answer with a suitable flow network example.

MODULE 5 — Approximation & Randomized Algorithms

SHORT ANSWER — 15 QUESTIONS

1. Define approximation algorithms? Explain the need for approximation algorithms in solving NP-hard optimization problems.
2. Differentiate between optimization problems and approximate solutions. Give a suitable example to illustrate the difference
3. Explain how approximation algorithms are useful for NP-hard problems. Discuss the trade-off between solution quality and computational efficiency.
4. Define approximation ratio. Explain how the approximation ratio is calculated for a minimization problem
5. Explain the 2-approximation algorithm for the Vertex Cover problem. Write its basic steps or pseudocode
6. Consider a graph with edges {(A,B),(B,C),(C,D),(D,E)}. Apply the 2-approximation algorithm for Vertex Cover and determine the vertex cover obtained. Explain why the solution satisfies the 2-approximation guarantee.
7. Define a randomized algorithm. Differentiate between deterministic and randomized algorithms with respect to their execution and output.
8. Explain the two major types of randomized algorithms. Give one characteristic of each type
9. A fair six-sided die is rolled once. Let X denote the number obtained. Determine the probability that X is greater than 4 and state the expected value of X.
10. Define Polynomial Identity Testing? Explain how randomization can be used to determine whether a polynomial is identically zero.
11. Explain the randomized Quick Sort algorithm. Why does random selection of the pivot improve its expected performance?
12. Apply Randomized Quick Sort to the array [8, 3, 7, 4, 9, 2]. Assume that 7 is selected as the first pivot. Show the first partition and state the next recursive subarrays.
13. State the Schwartz-Zippel Lemma and explain its significance in Polynomial Identity Testing
14. Explain the basic functionality of the Schwartz-Zippel Lemma with respect to a non-zero polynomial and random evaluation points.
15. A non-zero polynomial has total degree 4. If each variable is independently selected from a set of 20 values, use the Schwartz-Zippel Lemma to determine the maximum probability that the polynomial evaluates to zero.

ESSAY — 10 QUESTIONS

1. Explain optimization problems and the role of approximation algorithms in solving NP-hard optimization problems. Discuss the characteristics of an approximate solution and the importance of performance guarantees.
2. Analyze the need for approximation algorithms for NP-hard problems. Explain the concept of approximate solutions and performance guarantees with a suitable example
3. Describe the 2-approximation algorithm for Vertex Cover in detail. Provide its pseudocode, explain its operation using an example, and analyze its approximation guarantee
4. Analyze the 2-approximation algorithm for Vertex Cover. Prove its approximation ratio and demonstrate the analysis using a suitable example.
5. Explain randomized algorithms in detail. Compare deterministic and randomized algorithms, discuss Las Vegas and Monte Carlo algorithms, and explain their advantages and applications
6. Explain the basic probability concepts required for analyzing randomized algorithms. Discuss sample space, events, probability, random variables, expected value, and probability distributions with suitable examples.
7. Explain the randomized approach to Polynomial Identity Testing. Describe the algorithm, its working, and the probability of error. Illustrate the approach with a suitable example.
8. Describe Randomized Quick Sort with its algorithm and pseudocode. Explain its working using an example and discuss its expected worst-case running time. (Proof is not required.)
9. Explain the Schwartz-Zippel Lemma in detail. State the lemma, explain its basic functionality, and discuss how it is used in Polynomial Identity Testing. Illustrate the concept with a suitable example.
10. Analyze the probability of error in Polynomial Identity Testing using the Schwartz-Zippel Lemma. A non-zero polynomial has total degree 6, and each variable is chosen uniformly from a set of 30 values. Calculate the error probability and explain

how repeated testing can improve reliability.